v1.3.2
20260324

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module-2.1]
ASYNCHRONOUS PROGRAMMING

ASYNCHRONOUS PROGRAMMING

- A style of programming getting more and more important today
  - generally speaking, it's about about the execution and management of *asynchronous computations/request/processes*
    - abstracting from threads…
  - recently supported by all major programming frameworks and mainstream platforms
    - Java, .NET, iOS, Android..
  - including a variety of mechanisms and architectures
    - including event-driven programming, but not limited to
- Still an evolving issue
  - e.g. Microsoft case: first proposing Asynchronous Programming Model (APM), deprecated then for the Event-based Asynchronous Pattern (EAP) with .NET 2.0, deprecated for the Task-based Asynchronous Pattern (TAP) with .NET 4.5...
  - https://docs.microsoft.com/en-us/dotnet/standard/asynchronous-programming-patterns/

CURRENT ASYNC PANORAMA

- **Event-driven Programming**
  - event loop based control flow architectures
  - reactor pattern
- **Async Functions & Continuation Passing Style**
  - event handlers as continuations
  - promises library support
- **Async/await language mechanism**
  - mimicking synchronous being asynchronous
- **Coroutine-based concurrency**
  - lightweight threads

EVENT-DRIVEN PROGRAMMING

- A program is made of routines (***event handlers***) that are called (executed, triggered) when ***events*** occur
  - events can refer to any kind of (asynchronous) change of state or input from the environment which is relevant for the program
    - examples: user actions (mouse clicks, key presses), sensor outputs, messages, replies received…
  - *the control flow of the program depends on events*
    - non-determinism
  - "*programming without a call stack*"
    - after each handler execution the stack is empty
- Aspects
  - how handlers are associated to events
  - how events are received and managed
  - how handlers are executed

EVENTS VS. THREADS: AN OLD DEBATE

- Across different topics
  - *Why threads are a bad idea (for most purposes).* John Ousterhout - Invited Talk at the 1996 USENIX Technical Conference
  - *The Problem with Threads* (Edward Lee) - IEEE Computer, Vol. 39, No. 5, pp. 33–42, May 2006
  - *Why events are a bad idea* (for high-concurrency servers) (von Behren, Condit and Brewer) Proceedings of the 9th USENIX Conference on Hot Topics in Operating Systems, 2003
  - *Cooperative Task Management without Manual Stack Management - or, Event-driven Programming is Not the Opposite of Threaded Programming*. Atul Adya, Jon Howell, Marvin Theimer, William J. Bolosky, John R. Douceur. Microsoft Research. Proceedings of the 2002 Usenix Annual Technical Conference, Monterey, CA, June 2002.
- Each approach has pros and cons
  - the real challenge is about how to put them together…

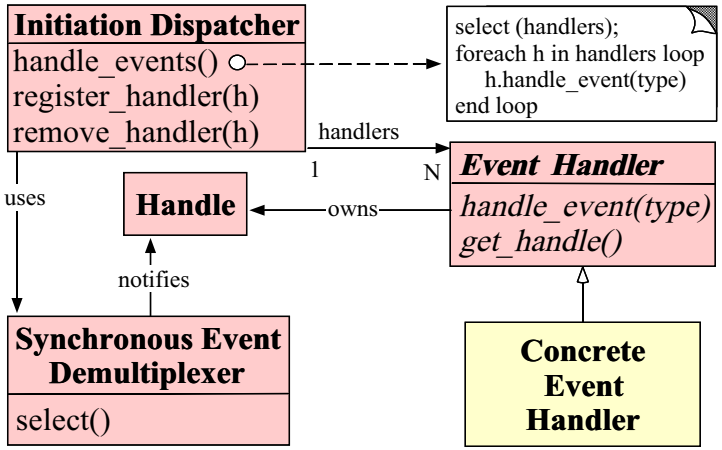
EXECUTION MODEL: “EVENT LOOP”

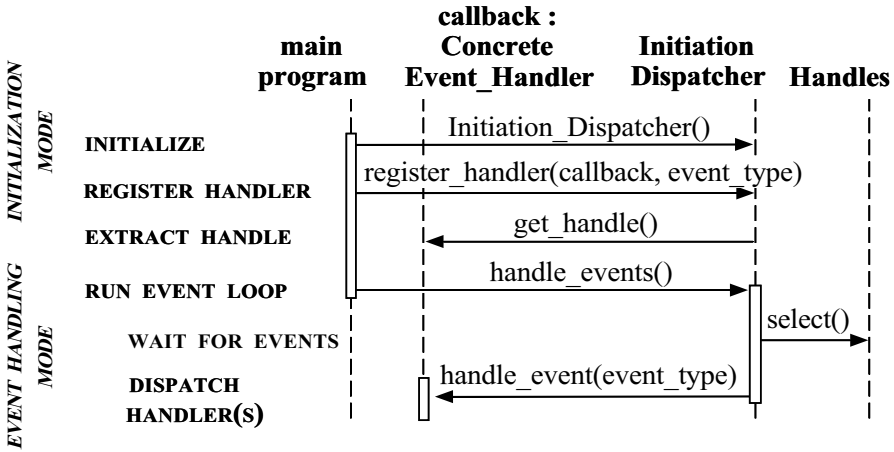
- **Event loop** control architecture (abstract model)
  - *single thread of control* waiting for events and executing the handler
  - an event queue used to keep track of events generated by either the environment or the event handler themselves

```
loop {
  Event ev = waitForEvent(eventQueue)
  Handler handler = selectHandler(ev)
  execute(handler)
}
```

- Event handlers are executed ***atomically***
  - if an event occurs while an handler is in execution, the corresponding event handler is executed only when the handler in execution has completed (no concurrency)

RELATED: REACTOR PATTERN





- Schmidt, Douglas et al. Pattern-Oriented Software Architecture Volume 2: Patterns for Concurrent and Networked Objects. Volume 2. Wiley, 2000.
- Schmidt, Douglas C., An Object Behavioral Pattern for Demultiplexing and Dispatching Handles for Synchronous Events - http://www.dre.vanderbilt.edu/~schmidt/PDF/reactor-siemens.pdf

MULTIPLE EVENT LOOPS

- Conceptually, *an event-loop can be used as the control architecture of a single active component*
  - event-driven behaviour
  - private event queue & asynchronous non blocking tasks
- A system can be composed by *multiple active components each one with its own event-loop*
  - not sharing memory
  - interacting by generating events

NEVER-BLOCKING RULE & CONCURRENCY

- The execution model leads to an important rule in designing/ programming event handlers:

> *event handlers should never blocks (should not contain blocking calls) and should always terminate (must not contain infinite loops)*

- This is to guarantee *responsiveness* of the system
  - a blocking call or infinite loop would block the event loop serving the events and events in the queue would no be processed
  - a blocking call must be replaced *by an asynchronous request or computation*, which will eventually generate some event in the future (result or error)
    - asynchronous requests/tasks are served by other threads, independent from the event loop
    - these threads interact with the event loop by inserting events in the event queue

FROM EVENT HANDLERS TO ASYNC FUNCTIONS & CALLBACKS

- Given the no-blocking rule, a main issue is:
  - *how to execute long-running handlers, without blocking and tampering reactivity?*

  => by executing *asynchronous* tasks / functions
    - the handler triggers the execution of an async task/function and goes on, without waiting for the result/error

- Key point/problem

  how to process results or errors that are asynchronously generated by the async task / function execution?

=> *callback* model

CALLBACK MODEL & CONTINUATION PASSING STYLE

- A function (***callback***) is specified as an argument (last argument) of the async task / function call
- The callback is called by the event loop when (if) processing the event related to the result or error generated by the async task / function execution
  - it is a kind of “implicit” handler registered dynamically to the event corresponding to the result/error
- In this way the callback defines a “*continuation*” of the computation

  => **Continuation-Passing Style (CPS)**

CONTINUATION PASSING STYLE (CPS)

- In functional programming, continuation-passing style (CPS) is a style of programming in which control is passed explicitly in the form of a *continuation*
  - first introduced by Gerald Jay Sussman and Guy L. Steele, Jr. 1975 working on Scheme
- Popular intermediate format for compilers of function languages,
  - many control flow constructs can be elegantly expressed
  - easy tail call optimization
- “Stack-less” programming

CPS IN GENERAL

- A function written in continuation-passing style takes an extra argument, a function of one argument i.e. *the continuation*
- When the CPS function has computed its result value, it "returns" it by calling the continuation function with this value as the argument
  - so the stack is not used, actually
- when invoking a CPS function, the calling function is required to supply a procedure to be invoked with the subroutine's "return" value

```
void sum(x, y, cont){
 cont(x + y);
}

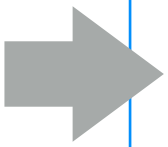
sum(1, 3, (res) -> {
  print(res)
});
```

CALLBACKS AS CONTINUATIONS

- *callbacks* are like continuations that are called when the result of the asynchronous computation is ready
  - examples:
    - the response of an asynchronous request to a remote service has arrived
    - the result of a I/O task is available
    - …
- Abstract example in JavaScript:

```
// synch version

function loadUserPic(userId) {
  let user = findUserById(userId);
  return loadPic(user.picId);
}
```



```
/* async version -
   both findUserById & loadUserPic
   are async processes */

function loadUserPic(userId, ret) {
  findUserById(userId, (user) => {
    loadPic(user.picId, ret);
  });
}

loadUserPic('john', (pic) => {
  ui.show(pic);
});
```

USING CLOSURES AS CALLBACKS

- Callbacks are often implemented as *closures*
  - technique for implementing lexically scoped name binding in a language with first-class functions.
  - operationally, a closure is a record storing a function[a] together with an environment.
    - the environment is a mapping associating each free variable of the function with the value or reference to which the name was bound when the closure was created

CALLBACK EXECUTION MODEL

- A main question is:

  who invokes the continuation/callback?

- Two main possibilities:
  1. a thread of control *different from* the thread that triggered the request (e.g., the thread generating the event)
     – problems: races
  2. in event-driven architecture based on the event loop model, *the continuation is executed by the event loop, i.e. the logical thread of control that triggered the async task / function* (which is different from the thread that *served/executed* the async task / function)
     - execution model of modern web apps (client/server)

CALLBACKS AS CONTINUATIONS EXECUTED BY THE EVENT LOOP

- In this case:
  - events are implicit and refer to the completion with success or error of an asynchronous request/computations
  - event handlers are represented by continuation functions specified when invoking the asynchronous request/computation
  - *closures* are typically used to define the context to be used when processing the event
- Concrete examples - web app development / frameworks
  - client side: Javascript, DART
  - server side: node.js, vert.x, any reactor-based framework

JAVASCRIPT - CLIENT EXAMPLE

- Event-loop ruling script execution
  - in web apps, events concern the web page life-cycle, the DOM, the user input, network interactions, … any event as defined by the HTML5 standard
- Example
  - Non blocking I/O

```
…
let ajaxRequest = new XMLHttpRequest;
ajaxRequest.open('GET',url);
ajaxRequest.send(null);
ajaxRequest.onreadystatechange =
function() {
  // what to do next …
}
```

  - wrong way of doing it…

```
// this is wrong
let ajaxRequest = new XMLHttpRequest;
ajaxRequest.open('GET',url);
ajaxRequest.send(null);
while (ajaxRequest.readyState === XMLHttpRequest.UNSENT){}
// !DEADLOCK!
```

JAVASCRIPT - CLIENT EXAMPLE

- `setTimeout(callback,timeout)` is another asynchronous function
  - registering a callback to be executed when the timeout expires
- What is the output of the following script?

```
for (let i = 1; i <= 3; i++){
  console.log("setting timer..." + i);
  setTimeout(() => { console.log(i); }, 0)
}
```

EXAMPLE IN NODE.JS

```
var sys = require(“sys”),
    http = require(“http”),
    url = require(“url”),
    path = require(“path”),
    fs = require(“fs”);

http.createServer((request, response) => {
  let uri = url.parse(request.url).pathname;
  let filename = path.join(process.cwd(), uri);
  path.exists(filename, (exists) => {
    if(exists) {
      fs.readFile(filename, (err, data) => {
        response.writeHead(200);
        response.end(data);
      });
    } else {
        response.writeHead(404);
        response.end();
    }
  });
}).listen(8080);

sys.log(“Server running at http://localhost:8080/”);
```

CONTINUATION-BASED ASYNCHRONOUS PROGRAMMING: KEY POINTS

- Using **a single thread to coordinate and manage multiple asynchronous task**
  - eventually executed by external/OS-backed threads
  - low memory footprint compared to multi-threaded solutions (where a stack is used for each thread)
  - based on an implicit event-loop
- **No low-level races**
  - no shared state, only one thread accessing the state
- **No low-level deadlocks**
  - async handlers never block

ASYNC CONTROL FLOW PATTERNS

- Sequence task1 ; task2

```
asyncTask1(…, (res1) -> {
  asyncTask2(…, (res2) -> {
    …}}
```

- Parallel task1 | task2

```
…
asyncTask1(…, (res1) -> {
 …
}
asyncTask2(…, (res2) -> {
 …
}
```

ASYNC PROGRAMMING: PROBLEMS

- Besides the benefits, there are some main well-known problems affecting CPS and event-driven programming, often referred altogether as *callback hell*
  - **asynchronous spaghetti**
    - overall computation fragmented into asynchronous handlers
    - this tampers the modularity and structure of programs
      - making them difficult to understand and maintain
  - **pyramid of doom / callback hell**
    - composition based on CPS leads to nested callbacks/ continuations
    - this increases complexity and leads to poor source code readability, bad reusability, bad extensibility

PYRAMID OF DOOM | CALLBACK HELL

- Callback nesting

```
step1(function(result1){
  step2(function(result2){
    step3(function(result3){
      // and so on …
    }
  }
}
```

"*I love async, but I can't code like this*"
(a complaint of a developer on the Node.js Google Group)

PYRAMID OF DOOM

- In previous examples…

```
function loadUserPic(userId, ret) {
  findUserById(userId, (user) => {
    loadPic(user.picId, ret);
  });
}

loadUserPic('john', (pic) => {
  ui.show(pic);
});
```

```
// Node.js

var sys = require("sys"),
    http = require("http"),
    url = require("url"),
    path = require("path"),
    fs = require("fs");

http.createServer((request, response) => {
  var uri = url.parse(request.url).pathname;
  var filename = path.join(process.cwd(), uri);
  path.exists(filename, (exists) => {
    if(exists) {
      fs.readFile(filename, (err, data) => {
        response.writeHead(200);
        response.end(data);
      });
    } else {
        response.writeHead(404);
        response.end();
    }
  });
}).listen(8080);

sys.log("Server running at http://localhost:8080/");
```

PROMISES

- The callback hell can be partially solved by adopting the ***promise*** mechanism
  - originally proposed in 1976 by Daniel Friedman and D. Wise, as a proxy object that represents an unknown result that is yet to be computed (similar to futures)
- Promises represent the *eventual completion and result of a single asynchronous operation*
  - they encapsulate asynchronous actions, acting much like a value returned from the result of a computation only that the value may not be available at the time
  - a promise can be rejected or resolved once and only once
    - immutable once-resolved promises
- As a key property, **they allow for flattening callback nesting**

PROMISES IN JAVASCRIPT

- Defined in Javascript standard (*)
  - also called *thenables*
    - a developer uses *then* to attach callbacks to a promise when it is either fulfilled or an exception is realised.
- Supported by different frameworks (with some different semantics...)
  - Q - Most complete & mature,
  - When.js - Lightweight
  - jQuery Promises (not fully conform)
- Example

```
let promisedPic = loadUserPic('john');

promisedPic.then((pic) => {
  ui.show(pic);
});
```

(*) now in ECMA-262 12th Edition - ECMAScript Language Spec (June 2021)
https://www.ecma-international.org/publications-and-standards/standards/ecma-262/

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

GENERAL CASE

- An operation that may take some time return a Promise object at once and then complete its task in background (async):

```
let myPromise = asyncFunc(args)
```

- Promise is regarded as being a Promise to deliver a future value => we can configure the Promise to specify what should happen when the asyncFunc is complete

```
myPromise.then(onComplete,onError)
```

  - onComplete => callback called if/when the asyncFunc is complete
  - onError (optional) => callback called if/when the asyncFunc produced an error

THE PROMISE OBJECT

- A Promise object can be in one of three states:
  - **pending**
    - when the Promise is first created
  - **resolved** (or *fulfilled*)
    - if the async function / task ends correctly
  - **rejected**
    - if the async function / task ends with an error
  - it is also called **settled** if it isn't pending
    - i.e. it is either resolved or rejected
- Remarks
  - once a Promise is settled, its state and value does not change

THE PROMISE API IN JAVASCRIPT

- Creating a promise

```
let promise = new Promise((resolve, reject) => {
  // do a thing, possibly async, then…

  if (/* everything turned out fine */) {
    resolve("Stuff worked!");
  } else {
    reject(Error("It broke"));
  }
});
```

- Attaching the callback

```
promise.then((result) => {
  console.log(result); // "Stuff worked!"
},(err) => {
  console.log(err); // Error: "It broke"
});
```

EXAMPLE

- Definition of an async function returning a Promise

```
function delayWithRandom(t,p){
  let promise = new Promise(
   (resolve,reject) => {
      setTimeout(() => {
        let r = Math.random();
        if (r > p) {
          resolve(r);
        } else {
          reject(r);
        }
      },t);
  });
  return promise;
}
```

- Calling the async function

```
let promise = delayWithRandom(1000,0.5);
promise.then((r) => {
  console.log("success: ",r);
},(r) => {
  console.log("fail: ",r);
});
```

PROMISE CHAINING

- Key feature for flattening the pyramid of doom
- *then* method returns itself a promise, so that promises can be chained

```
let promise = new Promise(
 (resolve, reject) => {
    resolve(1);
  });

let p1 = promise.then((val) => {
  console.log(val); // 1
  return val + 2;
});

p1.then((val) => {
  console.log(val); // 3
});
```

```
let promise = new Promise(
 (resolve, reject) => {
    resolve(1);
  });

promise.then((val) => {
  console.log(val); // 1
  return val + 2;
}).then((val) => {
  console.log(val); // 3
});
```

PROPAGATING VALUES

- The value of each Promise in a chain is whatever the previous onComplete/onError returns

```
let myProm1 = delayWithRandom(1000,0.5);
let myProm2 = myProm1.then((v) => {
  console.log(v);
  return “done”;
});
myProm2.then((v) => {
  console.log(v);
});
```

CHAINING ASYNC FUNCTIONS

- To chain async functions (not only the first generating the Promise), *we can return Promises* in the chain

```
function delayWithRand(t){
  let promise = new Promise(
    (resolve,reject) => {
      setTimeout(() => {
        var r = Math.random();
        resolve(r);
      },t);
  });
  return promise;
}
```

```
let promise = delayWithRand(1000);
promise.then((r) => {
  console.log("success: ",r);
  return delayWithRand(1500);
}).then((r) => {
  console.log("success: ",r);
  return delayWithRand(500);
}).then((v) => {
  console.log(v);
})
```

SEQUENTIAL EXECUTION FLOW PATTERN

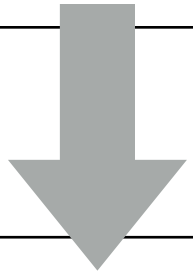
- Sequential execution flow pattern made easier

```
let myProm1 = asyncFunc1(…)
   .then((v) => {
      console.log(v);
      return asyncFunc2(…);
   }).then((v) => {
      console.log(v);
      return asyncFunc3(…);
   }).then(…);
```

FLATTENING THE PYRAMID

- Flattening the pyramid

```
findUserById(‘john').then((user) => {
  return findPic(user.picId).then((pic) => {
    ui.show(pic);
  });
});
```



```
findUserById('john')
  .then((user) => {
    return findPic(user.picId);
  })
  .then((pic) => {
    ui.show(pic);
  });
```

PARALLEL EXECUTION FLOW PATTERN

- Parallel execution flow pattern

```
let myPromise = asyncFunc1(…);

myPromise.then((v) => {
   console.log(v);
   return asyncFunc2(…);
});

myPromise.then((v) => {
   console.log(v);
   return asyncFunc3(…);
});
```

- Both the callbacks are called when asyncFunc1 ends, with the same value hold by the Promise

PROMISIFYING EXISTING CODE

- (Example taken from http://www.html5rocks.com/en/tutorials/es6/promises/)

```
function get(url) {
  // Return a new promise.
  return new Promise((resolve, reject) => {
    // Do the usual XHR stuff
    let req = new XMLHttpRequest();
    req.open('GET', url);

    req.onload = () => {
      // This is called even on 404 etc so check the status
      if (req.status == 200) {
        // Resolve the promise with the response text
        resolve(req.response);
      } else {
        // Otherwise reject with the status text which will hopefully be a meaningful error
        reject(Error(req.statusText));
      }
    };
   // Handle network errors
    req.onerror = () => {
      reject(Error("Network Error"));
    };
   // Make the request
    req.send();
  });
}
```

```
// configuring the promise..
get('story.json').then((response) => {
  console.log("Success!", response);
}, function(error) {
  console.error("Failed!", error);
});
```

COMBINING PROMISES - ALL

- Composing a set of independent asynchronous calls
  - to be executed in parallel, with a join point
- JavaScript Promise API: **all** method

```
let totalProm = Promise.all(promiseList);
totalProm.then(function(values){…})
```

- *the callback is called when all Promises are resolved or at least on is rejected*
- Example

```
let myProm1 = delayWithRand(1000);
let myProm2 = delayWithRand(2000);

let myTotalProm = Promise.all([myProm1,myProm2]);
myTotalProm.then((values) => {
  console.log(values[0]);
  console.log(values[1]);
});
```

COMBINING PROMISES - RACE

- Composing a set of independent asynchronous calls, but completing as soon as one completes
- JavaScript Promise API: **race** method

```
let totalProm = Promise.race(promiseList);
totalProm.then(function(value){…},
               function(value){…});
```

- *the callback is called when a Promise is resolved or rejected*
- Example

```
let myProm1 = delayWithRand(1000);
let myProm2 = delayWithRand(2000);

let myTotalProm = Promise.race([myProm1,myProm2]);
myTotalProm.then((value) => {
  console.log(value);
});
```

PROMISE ERROR HANDLING

- Significant improvement wrt callbacks
- Rules
  - any handler that returns a value and does not throw an error passes on success to the next Promise, including onFailure handlers
  - any handler that throws an exception passes a reject on the next Promise
  - an unhandled state is passed to the next Promise in the chain
    - if there is no rejection handler and the Promise is rejected then the state is passed to the next handler in the chain
    - once the state has found a handler then processing resumes its normal handler in the chain
    - similar to the synchronous case

IMMEDIATELY RESOLVED PROMISE

- `Promise.resolve` API to build promises that can be immediately resolved
  - then can be used to chain a callback to be executed in next cycle of the event loop

```
console.log("this cycle");
Promise
.resolve("done")
.then(() => {
  console.log("next cycle");
});
```

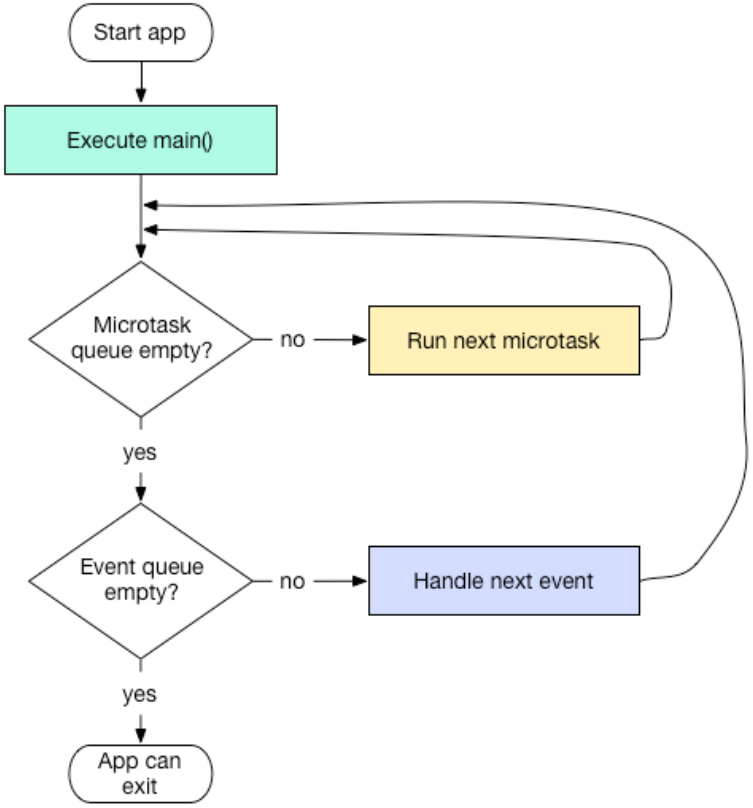
TASKS AND MICROTASKS

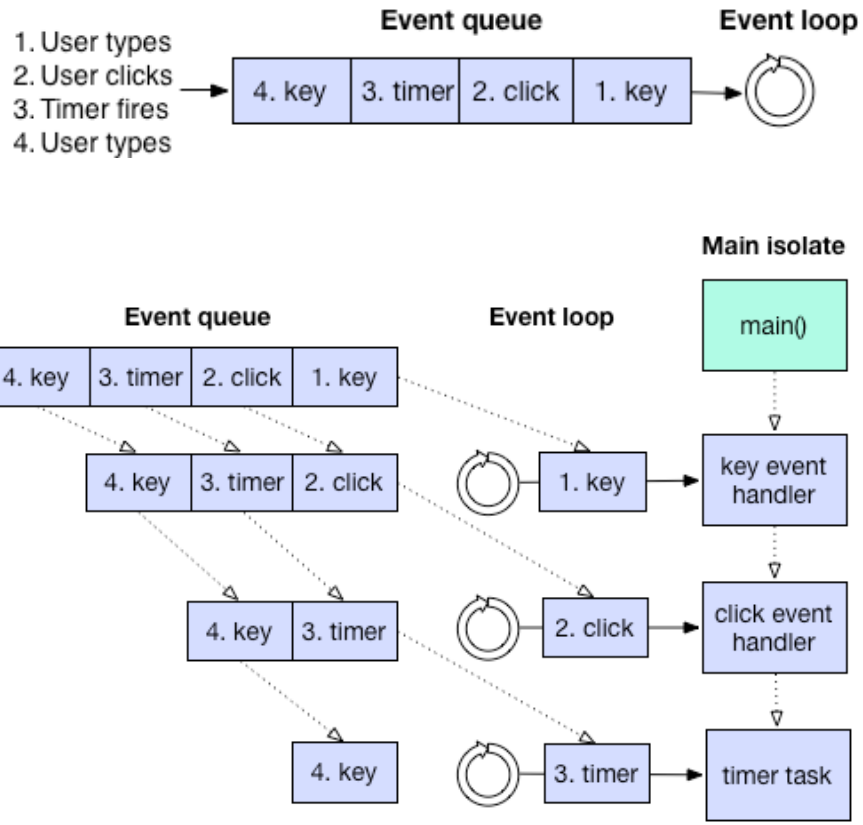
- In JavaScript and Web world, the event queue - also called dispatch queue - hosts both tasks and *microtasks*
  - event loop works as usual
    - takes tasks one at a time and allows the task to run to completion before the next task is serviced
    - tasks can also be preferentially dequeued (=> priorities)
  - micro tasks have been introduced as a lightweight version of the task
    - key feature: they are guaranteed to be executed before the next task starts
    - the microtask queue is processed after each task starts i.e. as soon as possible
- API do add micro tasks:

```
Promise.resolve().then(() => { … });
```

TASKS AND MICROTASKS - EVENT LOOP

(from https://www.dartlang.org/articles/event-loop/)





While the event loop is executing tasks from the microtask queue, the event queue is stuck: the app can't draw graphics, handle mouse clicks, react to I/O, and so on.

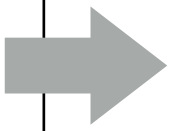
THE PROBLEM WITH PROMISES

- *Eagerness* and then parameter problem
  - we cannot pass parameters to functions used in a then
  - promises are ***eager***: when they are constructed, they immediately begin work

```
function delay(t){
  let promise = new Promise(
    function (resolve,reject){
      setTimeout(() => { resolve(); },t);
  });
  return promise;
}

function printTime(){
  var time = new Date().getTime();
  console.log(time);
}

printTime();
delay(1000)
.then(printTime)
.then(delay(1000))
.then(printTime);
```



```
…

printTime();
delay(1000)
.then(printTime)
.then(() => {
   return delay(1000);
 })
.then(printTime);
```

THE PROBLEM WITH PROMISES

- Cannot be used with loops/iterations

```
function delay(t){
  let promise = new Promise(
    (resolve,reject) => {
      setTimeout(function(){
        resolve();
      },t);
  });
  return promise;
}

for (let i = 0; i < 3; i++){
  delay(1000).then(() => {
    let time = new Date().getTime();
    console.log(i + " > " + time);
  })
}
```

it doesn't work as expected

THE PROBLEM WITH PROMISES

- Cancellation
  - promises cannot be cancelled
  - no control over the async task
- Possible waste of resources

FROM PROMISE TO ASYNC/AWAIT

- **async** & **await** language extension
  - most recent and promising solution in async programming
  - goal: *providing a synch programming style while keeping an asynchronous core*
- Support
  - introduced in ES2017 (the 8th edition of the JavaScript Spec, released in June 2017)
  - implemented in Microsoft .NET TAP (Task-based Asynchronous Pattern)
    - can be used in C# for instance
  - Implemented in an increasing number of languages
    - Python, DART, … language
    - Up-to-date list in https://en.wikipedia.org/wiki/Async/await

ASYNC WITH SYNCH STYLE

- Async style promoted by Promises

```
let p = asyncFunc(…);
p.then(onCompleteFunc, onErrorFunc)
```

- Main async-await idea: *reproducing a synchronous programming style, while keeping the asynchronous semantics*

```
let p = asyncFunc(…);
let res = await p;
<use res>
```

i.e., *conceptually wait for asyncFunc result (yielding the control) and then continue with the program (getting back the control) when the promise is solved*

AWAIT

- await operator

  `await <promise>`

  - can be applied to any Promise and *it suspends the function execution until the Promise is fulfilled*
  - **it does not block the control flow**
    - the control flow is *yield,* saving the current computation state

```
…
console.log("here1");
await delay(3000);
console.log("here2");
await delay(2000);
console.log("here3");
…
```

AWAIT - ONLY INSIDE ASYNC

- await *can be used only in the body of functions labelled as* **async**

```
async function f(){
  …
}

async function f1() {
 …
 await f();
 …
}
```

```
var myFunc = async function() {
  console.log("here1");
  await delay(3000);
  console.log("here2");
  await delay(2000);
  console.log("here3");
  return 1+2;
}
```

ASYNC FUNCTIONS

- **async** functions are suspendible functions returning a promise

```
var myFunc = async () => {
  return 1 + 2;
};
```

- The function is converted into an AsyncFunction object function
  - the function is just defined, not executed
- By calling the function, the code is executed as a normal function (in the same cycle) returning *a promise*

```
console.log("pre");
console.log(myFunc);
console.log(myFunc());
console.log("post");
```

OUTPUT:

```
pre
[AsyncFunction]
Promise {}
post
```

- AsyncFunction object contains the low-level machinery (based on *co-routines,* discussed later) to allow for expressing the body of the asynchronous function using a synchronous style

ASYNC FUNCTION WITH/WITHOUT AWAIT

- Note that async function can be called either with await or not, producing a different behaviour
- Example

```
async function pause(t){
  console.log("before");
  let promise = new Promise(
    (resolve,reject) => {
      setTimeout(() => {
        resolve();
      },t);
  });
  await promise;
  console.log("after");
}
```

without

```
console.log("before call");
pause(1000);
console.log("after call");
```

with

```
async function main(){
  console.log("before call");
  await pause(1000);
  console.log("after call");
}
main();
```

AWAIT: DETAILS

- `await <promise>` converts a Promise into a result or an exception *just as if it was a result from a synchronous function*
  - if the Promise is resolved, the await operator extracts the resolved value and returns it
  - if the Promise is rejected then the await throws an exception with the rejected value

AWAIT AND CO-ROUTINES

- await behave likes a return that can be resumed
  - when a function executes an await, the thread returns to the calling program which receives the Promise object created by the async function in a pending state
  - the state of the computation is saved, to be restored when the control is back
  - the async function is resumed asynchronously, as soon as the event-loop thread fetches the corresponding event in the event queue
- To release and resume the control in the implementation of await/ async, the **co-routine** mechanism is exploited
  - old mechanism adopted for cooperative scheduling
  - brought back in asynchronous programming based on a single-control flow
  - more at the end of this module

BENEFITS

- Asynchronous programming greatly simplified
  - no need to explicitly use Promises/then/chains for implementing the sequence execution pattern
    - eagerness & then parameter problem solved
    - can be used with loops/iterations…

```
async function waitFor(t){
  console.log("waitFor - pre");
  await delay(t);
  console.log("waitFor - post");
}

async function main(){
  console.log("Before");
  for (let i = 0; i < 3; i++){
    await waitFor(1000);
    console.log("Step "+i);
  }
  console.log("After");
};
```

ISSUES

- Some issues affecting async/await
  - await can be used only inside an async function body => it cannot be used at the top level
    - need to introduce at least one main async function
  - an async function cannot resume while the thread is busy running other handlers
    - the event-loop semantics is preserved
  - **blocks {…} are no more guaranteed to be atomic**
    - the execution of the code of a single block {…} can span over multiple iterations of the event loop
    - => race conditions
  - composing await/async => Need to mix await/async with Promises API

COMPOSITION ISSUE: EXAMPLE

- Composing await/async => Need to mix await/async with Promises API

```
async function f(){
  await delay(1000);
  return 1+2;
}

async function g(){
  await delay(5000);
  return 2+3;
}

async function compose(){
  console.log("before");
  let r = await Promise.all([f(),g()]);
  console.log("done: "+r);
}

compose();
```

“DESIGN CLASH” ISSUE

- Design clash between synchronous and asynchronous
  - strong discipline needed in mixing synch/asynch style to avoid programs whose behaviour is difficult to understand
  - *if we want to mix/integrate them, what are the key abstraction concepts / rationale that we can use to think /design a program, besides the programming mechanisms?*
  - *What about modularity? What impact on reuse and extensibility?*
- Need of higher level of abstractions/concepts that naturally integrate a sync and an async perspective

MODULARITY/ENCAPSULATION ISSUE

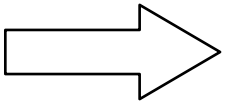
```
function delay(t){
  let promise = new Promise(
    (resolve,reject) => {
      setTimeout(function(){
        resolve();
      },t);
  });
  return promise;
}

async function f(){
  console.log("started f()")
  await delay(2000);
  console.log("done f()");
}

function g(){
  console.log("started g");
  f();
  console.log("done g");
}

g();
```

this does not work as expected



```
function delay(t){
  let promise = new Promise(
    (resolve,reject) => {
      setTimeout(function(){
        resolve();
      },t);
  });
  return promise;
}

async function f(){
  console.log("started f()")
  await delay(2000);
  console.log("done f()");
}

async function g(){
  console.log("started g");
  await f();
  console.log("done g");
}

g();
```

ABOUT COROUTINES

- Generalisation of the subroutine concept by allowing **execution** to be **suspended** and **resumed** [Con63]
  - for non-preemptive, cooperative multitasking
  - basic building block for implementing cooperative tasks, exceptions, event loops, iterators, infinite lists and pipes
  - not related to events and event-driven programming
- Historically, coroutines originated as an assembly language method, but are supported in some high-level programming languages.
  - early examples include Simula, Smalltalk, and Modula-2
- The mechanism has received a renewed attention in recent times mainly because of async programming
  - more recent examples are Ruby, Lua, Julia, Go, Kotlin
  - Up-to-date view: https://en.wikipedia.org/wiki/Coroutine

COROUTINES AS GENERALISED ROUTINE

- When calling a coroutine*, a coroutine can exit by calling other coroutines*, which may later return to the point where they were invoked in the original coroutine
  - from the coroutine's point of view, it is not exiting *but calling another coroutine*
  - calling another coroutine can be done by "yielding" to it

```
var q := new queue

coroutine produce
    loop
        while q is not full
            create some new items
            add the items to q
        yield to consume

coroutine consume
    loop
        while q is not empty
            remove some items from q
            use the items
        yield to produce
```

A cooperative one-flow based producer consumer system (from https://en.wikipedia.org/wiki/Coroutine)

COROUTINES CHARACTERISTICS

- A coroutine instance holds state, and varies between invocations
  - there can be multiple instances of a given coroutine at once
- Traditional *subroutines* are special cases of coroutines
  - any subroutine can be translated to a coroutine which does not call yield
- Can be easily implemented in languages natively supporting continuations
  - it *cannot* be implemented just as a library on top of a language based on just subroutines and subroutine-like mechanism (e.g. methods)

CONCURRENCY MODEL BASED ON COROUTINES

- Coroutines can be used as low-level mechanism to implement *lightweight threads*
  - one thread can have many coroutines inside it, but only one coroutine is in execution at a given time
- Fast and cheap to create and to manage
  - not requiring context switching by the OS
- Kotlin language example - main concepts:
  - Suspending computations
  - Coroutine dispatchers
  - Coroutine builders

A TOY EXAMPLE

```
import kotlinx.coroutines.*

fun main(args: Array<String>) = runBlocking {
    println("before async call")
    val result = async {
        println("inside the async call")
        delay(1000)
        println("exiting the async call")
        100
    }
    println("after the async call, before greet")
    greetDelayed(200)
    println("after greet trigger")
    println("${result.await()} ")

}

suspend fun greetDelayed(delayMillis: Long) {
    delay(delayMillis)
    println("Hello, World!")
}
```

OUTPUT

```
before async call
after the async call, before greet
inside the async call
Hello, World!
after greet trigger
exiting the async call
100
```

SUSPENDING COMPUTATIONS IN KOTLIN

- Computations that can suspend their execution without blocking the thread in which they reside
  - a suspending computation allows the thread to be used for other computations until they need to be resumed
- Types
  - **suspending functions**
    - suspending computations in the shape of a function: modifier suspend
  - **suspending lambda**
    - like a normal lambda, but can suspend its own execution by calling other suspending functions

SUSPENDING FUNCTIONS: EXAMPLE

- Example

```
suspend fun greetAfter(name: String, delayMillis: Long) {
      delay(delayMillis)
      println("Hello, $name")
 }
```

  - the execution of greetAfter() will be suspended when delay() is called
  - delay() is a suspending function itself, suspending the execution for a give duration
  - once delay() has been completed, greetAfter() will resume its execution normally
  - and while greetAfter() is suspended, the thread of execution may be used to do other computations

COROUTINE DISPATCHERS & BUILDERS

- **Coroutine dispatchers**
  - in order to decide what thread to start or resume a coroutine on a coroutine dispatcher is used.
- **Coroutine builders**
  - a function that takes a suspending lambda and creates a coroutine to run it
  - coroutine builders provided by Kotlin:
    - `async()` - start a coroutine when a result is expected
    - `launch()` - start a coroutine that doesn't return a result
    - `runBlocking()` - bridge blocking code into suspendable code

RELATED TO COROUTINE: FIBERS

- A **fiber** is a lightweight thread of execution
  - like threads, fibers share address space
  - differently from threads, fibers use *cooperative multitasking* while threads use preemptive multitasking
    - if threads typically depend on the kernel's thread scheduler to preempt a busy thread and resume another thread, fibers yield themselves to run another fiber while executing
      - so can be implemented using a single thread
- Fibers are like coroutines, but at different level of abstraction
  - coroutines are a language-level construct, a form of control flow
  - fibers are a systems-level construct, viewed as threads that happen to not run in parallel
  - fibers may be viewed as either an implementation of coroutines or as a substrate on which to implement coroutines
- Implemented by several OS/frameworks/languages
  - e.g. UNIX/pthreads, Windows/.NET
  - Java: **virtual threads** (from JDK 19)

VIRTUAL THREADS VS. ASYNC PROGRAMMING: A NEVER ENDING DEBATE

- See the recent overview by Brian Goetz about Virtual Threads **[Goe22]**
  - virtual threads provides a better modularity and encapsulation for modularising logical processes and control flows (vs. async programming…)
  - however virtual threads — like physical threads — do not provide a first-class treatment for *events* and reactive computations
    - must be implemented and managed at the application level..

REFERENCES

- Trevor Burnham. AsyncJavascript. The Pragmatic Programmers Series
- Event-based Concurrency Control. Tom Van Cutsem, Vrije Universiteit Brussel. Tutorial at OOPSLA 2009. Slides available at http://soft.vub.ac.be/~tvcutsem/talks/presentations/T37_nobackground.pdf
- Programming Without a Call Stack – Event-driven Architectures. Gregor Hohpe. www.eaipatterns.com
- MSDN - Asynchronous programming on .NET 4.5
- Stefan Tilkov and Steve Vinoski. 2010. Node.js: Using JavaScript to Build High-Performance Network Programs. IEEE Internet Computing 14, 6 (November 2010), 80-83.
- Ian Elliot. JavaScript Async: Events, Callbacks, Promises and Async Await. I/O Press
- EcmaScript
  - https://www.ecma-international.org/ecma-262/6.0/
  - https://www.ecma-international.org/ecma-262/8.0/

REFERENCES

- **[Con63]** Conway, M. E. (July 1963). "Design of a Separable Transition-Diagram Compiler". Communications of the ACM. 6 (7): 396–408.
- **[Mou04]** Ana Lucia de Moura; Roberto Ierusalimschy (2004). "Revisiting Coroutines". ACM Transactions on Programming Languages and Systems. 31 (2): 1–31. CiteSeerX 10.1.1.58.4017. doi:10.1145/1462166.1462167.
- **[Pro18]** Aleksandar Prokopec. Theory and Practice of Coroutines with Snapshots. ECOOP 2018
- **[Goe22]** Brian Goetz. Virtual Threads: New Foundations for High-Scale Java Applications. InfoQ (Sept. 22)
  - https://www.infoq.com/articles/java-virtual-threads/
- “Holy crap: JVM has coroutine/continuation/fiber etc.”
  - https://web.archive.org/web/20150319052055/http://weblogs.java.net/blog/forax/archive/2009/11/19/holy-crap-jvm-has-coroutinecontinuationfiber-etc